YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY) BALMATTA, MANGALORE

Project – High Level Design

on

Honeypot Network for Agriculture Threat Intelligence

SUBMITTED BY

Suhail PM- 23BCCE136

Industry Mentor: Mr.Shashank Kamoor

Guided by: Ms.Vijayalaxmi

DEPARTMENT OF COMPUTER SCIENCE

IN PARTIAL FULLFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF
BACHELOR OF COMPUTER APPLICATION/ BACHELOR OF COMPUTER SCIENCE

May - 2026

TABLE OF CONTENTS

1. Introduction
1.1 Purpose of the Document 3
1.2 Scope 3
1.3 Intended Audience 3
1.4 System Overview 3

2. System Design
2.1 Application Design 4
2.2 Process Flow 5
2.3 Information Flow 6
2.4 Components Design 6
2.5 Key Design Considerations 7
2.6 API Catalogue 8

3. Data Design
3.1 Data Model 9
3.2 Data Access Mechanism 9
3.3 Data Retention Policies 10
3.4 Data Migration 10

4. Interfaces 11

5. State and Session Management 12

6. Caching 13

7. Non-Functional Requirements
7.1 Security Aspects 14
7.2 Performance Aspects 14

8. References 15

1. Introduction

1.1 Purpose

This document presents the High Level Design (HLD) of the AgriCloud™ Honeypot System. It defines the overall system architecture, key components, data flow, and design considerations required to implement a cybersecurity honeypot in an agriculture-based environment.

1.2 Scope of the Document

The scope covers:

- Python TCP socket server (honeypot.py)
- Web-based Agriculture Smart Farm portal (HTML/CSS/JS)
- Fake portal trap pages for attacker deception
- Threat intelligence monitoring dashboard

It does not cover cloud deployment or production SIEM integration.

1.3 Intended Audience

- Academic examiners and faculty evaluating the project
- Industry mentors reviewing the technical design
- Students studying socket programming and threat intel
- Security professionals in Agriculture IoT environments

1.4 System Overview

The AgriCloud Honeypot is a multi-layered cybersecurity trap that simulates a legitimate Agriculture IoT portal.

Network Layer:

A Python TCP socket server on port 8888 that captures the IP address and timestamp of every connection attempt, responds with a fake system banner, and logs all activity to honeypot_log.txt.

Application Layer:

A multi-page web portal styled as a Smart Farm Management System with farmer login, registration, real IoT dashboard, fake trap page, and a SOC-style monitoring dashboard.

2. System Design

2.1 Application Design

Architecture Style : Layered / Deception-Based

Backend Language : Python 3.x (standard library only)

Frontend Technology : HTML5, CSS3, Vanilla JavaScript

State Persistence : Browser localStorage

Communication : TCP Sockets on port 8888

Deployment Target : Localhost — Educational use only

Log Storage : honeypot_log.txt + localStorage

```
ATTACKER
↓
TCP HONEYPOT SERVER (Python)
↓
Log File (honeypot_log.txt)
```

```
WEB PORTAL SYSTEM
┌──────────────────────────────────┐
│          Login Page              │
│          Real Dashboard          │
│          Fake Honeypot Page      │
│          Monitoring Dashboard    │
└──────────────────────────────────┘
↓
localStorage
```

2.2 Process Flow

NETWORK LAYER — honeypot.py

Step 1 Python script starts. TCP socket created. socket.socket(AF_INET, SOCK_STREAM)

Step 2 Socket option set to allow immediate port reuse. setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)

Step 3 Socket bound to all interfaces on port 8888. server_socket.bind(('0.0.0.0', 8888))

Step 4 Socket begins listening. Up to 5 queued. server_socket.listen(5)

Step 5 Main thread blocks waiting for connection. server_socket.accept()

Step 6 Connection arrives. Daemon thread spawned. threading.Thread(daemon=True)

Step 7 Thread sends fake AgriTech banner to attacker. client_socket.send(FAKE_BANNER.encode('utf-8'))

Step 8 IP address, port, and timestamp logged to file. open(LOG_FILE, 'a') → f.write(log_entry)

Step 9 Socket closed. Main thread returns to Step 5. client_socket.close() in finally block

WEB PORTAL — Attacker Path

Step 1 Attacker opens login.html → PAGE_VISIT logged to localStorage

Step 2 Wrong credentials entered (1st attempt) → LOGIN_ATTEMPT logged to localStorage

Step 3 Wrong credentials entered (2nd attempt) → LOGIN_ATTEMPT logged to localStorage

Step 4 Wrong credentials entered (3rd attempt) → ACCESS_DENIED logged to localStorage

Step 5 SESSION QUARANTINED overlay shown on screen

Step 6 Browser redirects to fake-page.html after 2 seconds window.location.href redirect

Step 7 Fake farm shows random live-looking sensor values

Step 8 Every button click → PERMISSION DENIED modal

Step 9 Dashboard polls localStorage every 800ms honeypot_dashboard.html updates live

2.3 Information Flow

TCP Connection Log
Source : Any TCP client on port 8888
Destination : honeypot_log.txt
Mechanism : Python file append

Web Login Event
Source : login.html failed logins
Destination : localStorage['honeypot_logs']
Mechanism : JSON array push

Dashboard Refresh
Source : honeypot_dashboard.html
Destination : Reads localStorage
Mechanism : setInterval every 800ms

Attacker Redirect
Source : login.html (3rd failure)
Destination : fake-page.html
Mechanism : window.location.href

2.4 Components Design

honeypot.py
Technology : Python 3 / socket
Role : TCP listener — logs IPs, sends fake banner
Port : 8888

test_connection.py
Technology : Python 3 / socket
Role : Simulates attacker connection for testing
Port : 8888

login.html
Technology : HTML / CSS / JS
Role : Fake Agriculture login with 3-attempt lockout
Medium : Browser

farm-details.html
Technology : HTML / CSS / JS
Role : Real farmer dashboard — IoT sensors, crops
Medium : Browser

fake-page.html
Technology : HTML / CSS / JS
Role : Honeypot trap — all actions denied
Medium : Browser

honeypot_dashboard.html
Technology : HTML / CSS / JS
Role : SOC monitoring dashboard — polls localStorage
Medium : Browser

honeypot.js
Technology : JavaScript
Role : Tracker library — logs events to localStorage
Medium : Browser

2.5 Key Design Considerations

Deception Authenticity
fake-page.html is visually identical to the real dashboard. Sensor values use Math.random() in realistic ranges and update every 3 seconds so data appears live.

Multi-threading
threading.Thread handles multiple attackers simultaneously. daemon=True ensures threads are cleaned up on exit.

Stateless Backend
Python server has no session state, no database, no command execution. Zero exploitable attack surface.

No External Dependencies
Only Python standard library used — socket, threading, datetime, os. No pip install needed.

localStorage Isolation
All web portal state is confined to the browser. No data transmitted over network from the web layer.

2.6 API Catalogue

start_honeypot()
Module : honeypot.py
Params : None
Action : Starts TCP server loop — never returns

handle_connection(socket, addr)
Module : honeypot.py
Params : client_socket, client_address tuple
Action : Sends banner, logs, closes socket

log_connection(ip, port, ts)
Module : honeypot.py
Params : ip string, port int, timestamp string
Action : Appends entry to honeypot_log.txt

sendLog(event, extra)
Module : honeypot.js
Params : event string, extra object (optional)
Action : Writes to localStorage and optional backend

attemptLogin(id, pass)
Module : login.html
Params : farmerId string, password string
Action : Validates credentials or increments lockout

generateFarmerId()
Module : login.html
Params : None
Action : Returns unique AC-KL/KA-XXXX format string

3. Data Design

3.1 Data Model

FILE STORE — honeypot_log.txt

```
timestamp   : String  — e.g. 2026-04-17 10:03:29
ip_address  : String  — padded to 20 chars    e.g. 127.0.0.1
port        : Integer — always 8888
status      : String  — LOGGED & DENIED
```

Sample log line:
[2026-04-17 10:03:29] CONNECTION ATTEMPT |
IP: 127.0.0.1 | Port: 8888 |
Status: LOGGED & DENIED

BROWSER STORE — localStorage key: honeypot_logs

```
id        : Integer  — timestamp-based ID
event     : String   — PAGE_VISIT / LOGIN_ATTEMPT /
                       ACCESS_DENIED / PAGE_EXIT
ts        : String   — ISO datetime
ts_local  : String   — locale formatted
url       : String   — page URL
browser   : String   — Chrome / Firefox / Safari / Edge
os        : String   — Windows / macOS / Linux / iOS
screen    : String   — e.g. 1920x1080
tz        : String   — e.g. Asia/Kolkata
user      : String   — attempted Farmer ID (login events)
```

3.2 Data Access Mechanism

Python writes using:
open(LOG_FILE, 'a') → append mode, never overwrites

JavaScript reads using:
JSON.parse(localStorage.getItem('honeypot_logs'))

JavaScript writes using:
localStorage.setItem(key, JSON.stringify(array))

Dashboard reads every 800ms using:
setInterval(pollLogs, 800)
Only new entries processed each cycle.

3.3 Data Retention Policies

honeypot_log.txt
Retained indefinitely. Append-only. Manual clear needed.

localStorage['honeypot_logs']
Retained until CLEAR button clicked or browser cleared.
Max 200 entries enforced by honeypot.js (MAX_LOCAL = 200).

localStorage['registered_farmers']
Retained until browser data is cleared. No expiry.

3.4 Data Migration

No data migration required. System designed for local lab sessions. Log files can be manually copied for reporting. No database schema or version-controlled data format exists.

4. Interfaces

Farmer Login Portal (login.html)
Audience : Attacker / Farmer
Features : Animated sunrise background, Farmer ID field, password field, 3-attempt lockout dots, live stats counter

Real Farm Dashboard (farm-details.html)
Audience : Legitimate Farmer
Features : 6 IoT sensor panels, crop status table, 7-day weather forecast, irrigation schedule

Honeypot Trap Page (fake-page.html)
Audience : Attacker (redirected)
Features : Identical to real dashboard, all buttons return PERMISSION DENIED, red intel panel

Threat Dashboard (honeypot_dashboard.html)
Audience : SOC Analyst / Instructor
Features : Live log table, threat gauge, activity chart, top source IPs, SIM ATTACK, CLEAR controls

Connection Log File (honeypot_log.txt)
Audience : SOC Analyst
Features : Plain text, aligned columns, one entry per TCP connection, machine-readable

Terminal Output (Python console)
Audience : Operator
Features : Real-time live output mirroring log file

Interface Interaction Sequence — Attacker Scenario

1 Attacker port-scans, finds port 8888, connects via TCP
→ Python logs IP. Attacker receives FAKE_BANNER.

2 Attacker opens login.html in browser
→ PAGE_VISIT event logged to localStorage.

3 Attacker enters guessed credentials three times
→ Red dots fill. ACCESS_DENIED logged on 3rd attempt.

4 Browser auto-redirects to fake-page.html
→ Attacker believes they have partial access.

5 Instructor opens honeypot_dashboard.html
→ All events shown. Threat gauge reads HIGH or CRITICAL.

User → Login → Invalid Credentials → Attempts Increase
→ Attempts ≥ 3 → Redirect to Fake Page
→ Track Activity → Store Logs → Display in Dashboard

5. State and Session Management

Python honeypot is fully STATELESS.
Each TCP connection is independent. No session tracking.

Web portal localStorage keys:

honeypot_logs
Used by : all pages + honeypot.js
Purpose : Shared event log — polled by dashboard

registered_farmers
Used by : register.html (set), login.html (read)
Purpose : Array of farmer credential objects

loggedInFarmer
Used by : login.html (set), farm-details.html (read)
Purpose : Current authenticated farmer ID

hp_attempt_count
Used by : login.html
Purpose : Failed login counter for current session

Session lifecycle — Legitimate Farmer:
1 Open login.html → no session key present
2 Enter valid credentials → validation passes
3 loggedInFarmer set → redirect to farm-details.html
4 Dashboard reads key → personalised display
5 Logout → loggedInFarmer removed

6. Caching

No server-side caching is implemented.

Python processes each connection independently with no result caching, connection pooling, or keep-alive.

Web frontend uses browser-native caching for static files.

Dashboard polls localStorage every 800ms. It reads the full array each cycle and computes only the delta from lastLogCount. No stale state — every event is captured.

In production: static assets would be cached at NGINX reverse proxy. Event log endpoint would be marked no-store to prevent caching of sensitive threat data.

7. Non-Functional Requirements

7.1 Security Aspects

Attack Surface Minimisation
Python server sends only a hardcoded ASCII string.
No command execution, no file access from socket input,
no database, no shell. Zero exploitable surface.

Deception Effectiveness
Fake farm portal uses Math.random() in realistic ranges.
Values update every 3 seconds. Identical visual styling
to real dashboard — attackers cannot detect the redirect.

Credential Security
Farmer login validation runs entirely in the browser
against localStorage. No passwords transmitted over network.

Farmer ID Uniqueness
Before issuing AGF-YYYY-XXXX, system checks all existing
IDs in localStorage. Regenerates on collision.

Ethical Containment
Binds to 0.0.0.0 but documented for localhost only.
No port forwarding configured. Safe for lab use.

7.2 Performance Aspects

Concurrent Connection Handling threading.
Thread handles many attackers simultaneously.
No shared mutable state between threads. O(1) per connection.

Queue Capacity
listen(5) queues up to 5 pending connections.
Adequate for 1–5 students in classroom demo.
Production: increase to 50–100.

Dashboard Polling
setInterval(800ms) reads localStorage once per cycle.
JSON.parse of 200 entries takes under 1ms.
DOM update is differential — only new rows added.

Log File Write Performance
open() + write() + close() per connection.
Each open is ~0.1–1ms OS syscall.
For >100 connections/sec, use buffered logger in production.

8. References

1. Python Software Foundation. socket library.
https://docs.python.org/3/library/socket.html

2. Python Software Foundation. threading library.
https://docs.python.org/3/library/threading.html

3. Python Software Foundation. datetime library.
https://docs.python.org/3/library/datetime.html

4. NIST. Guide to Intrusion Detection and Prevention Systems.
https://www.nist.gov/

5. OWASP Foundation. Honeypots and Deception Technology.
https://owasp.org/

6. CISA. Understanding Honeypots.
https://www.cisa.gov/

7. MDN Web Docs. Window.localStorage.
https://developer.mozilla.org/

8. Spitzner, L. (2003). Honeypots: Tracking Hackers.
Addison-Wesley Professional.

9. IBM SkillsBuild. (2026). Cloud Computing, Cyber Security
& Ethical Hacking Programme Materials.